

Project Work Week 2

You have to work on the following topics.

- 1) Process
- 2) Software Development
- 3) Ethics

Remember, you have to perform work on each topic and on **each** bullet point. Be done with the project work before we meet again next week's lecture.

1) Process

- From now on, organize your project work into issues (tickets) on GitHub. Describe each issue drawing inspiration from the guidelines presented in class.
 - Issues shall describe the tasks presented in the `README_PROJECT.md` files. Add additional issues for tasks that you identify to complete your project work.
 - Make sure that you label your issues according to their type, e.g., do they describe a new feature, an enhancement, a bug, etc.
- From now on, make sure that you record your work in multiple *small* commits. Not one big one per task, as suggested by Mark Seeman in the reading material.
 - Remember, to register all co-authors that contributed to a commit, see session 01.
 - Each co-author is registered on a separate line in the commit message
 - Make all these small commits in short-lived feature branches, which you merge with main (at least once per day).
- From now on, use a GitHub project board to organize your work and to make your work status visible.
 - Use the project template called “*Team planning*”, i.e., use the *board layout* (not the table or project layout).
 - Add all issues that you might have already created to the project board, this can be done manually or automatically.
 - Make sure that the status of the issues is properly reflected in the project board, i.e., move issues that are work-in-progress to the respective column, etc.

2) Software Development

The following are separate tasks. That is, treat them as such. Even though it might tempting to perform all the refactorings in one big chunk of work, train to work on separate concerns, one at a time.

2.a) Refactor *Bison* to use an external library for CSV file handling

In general, comprehensive support for CSV files with correct handling of escape characters, line breaks, etc. is quite difficult. In case you are using “manual” parsing and unparsing of comma-separated values when receiving and storing cheeps to and from disk, refactor your implementation of *Bison* so that it uses a library that handles CSV files properly. One of the most used libraries for handling CSV files in C# seems to be *CsvHelper*. Use this or a comparable library to read cheeps from and write cheeps to a CSV file.

For now, cheeps shall be objects that are instances of a respective `record` that was presented in class:

```
public record Cheep(string Author, string Message, long Timestamp);
```

2.b) Refactor data persistence to library project `SimpleDB`

Refactor the entire data persistence solution so that it resides in a library project `SimpleDB` on which your `Bison.CLI` project depends, see the official documentation on how to establish references between projects.

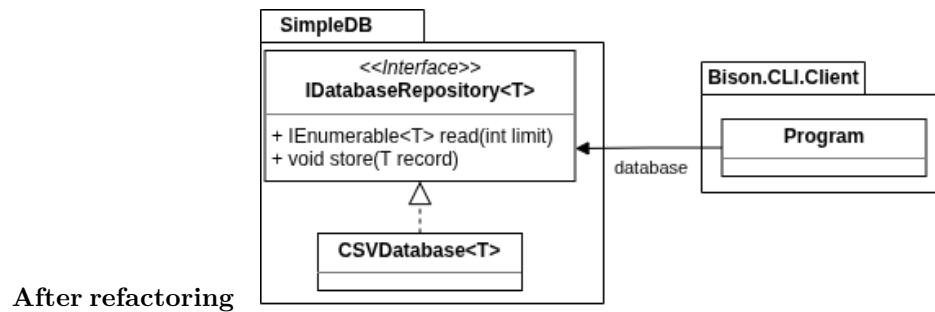
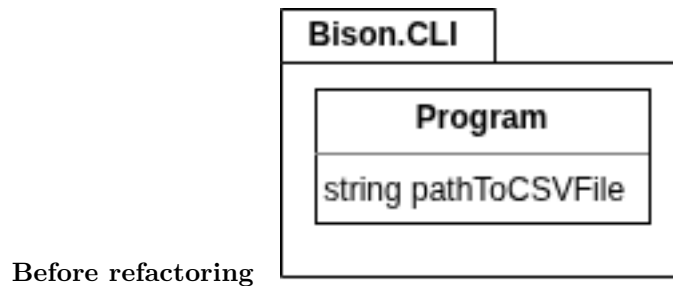
Create a new .NET classlib project called `SimpleDB` (or reuse the one from class) and create a class `CSVDatabase<T>` in it that implements the `IDatabaseRepository<T>` that was presented in the lecture. Note, it shall not be possible to create further sub-classes of class `CSVDatabase<T>`, see the official documentation on how to achieve that.

Make sure that your `SimpleDB` relies on the refactored code from the task above, i.e., that it depends on an external library for parsing and unparsing of CSV data.

Note, after this refactoring, your `Bison.CLI` project is likely quite “slim” and contains only code for retrieving command-line arguments and for printing output, i.e., it contains the application’s user interface.

When illustrated as UML class diagram, your current program (the one that you created in last week’s project work) looks likely as on the left below. That is, all functionality is contained in a single `Program` class, which is contained in your `Program.cs` file.

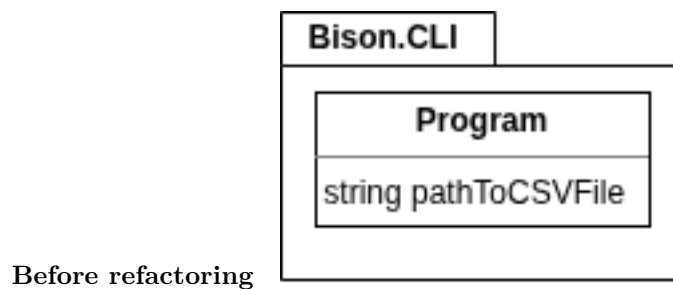
After the refactoring, it should look as on the right below. That is, a generic class `CSVDatabase<T>` implements the generic interface `IDatabaseRepository<T>` and the two methods `Read` and `Store`. A reference to the `database` is used in the main program class so that cheeps can be stored to the database and be retrieved from it.

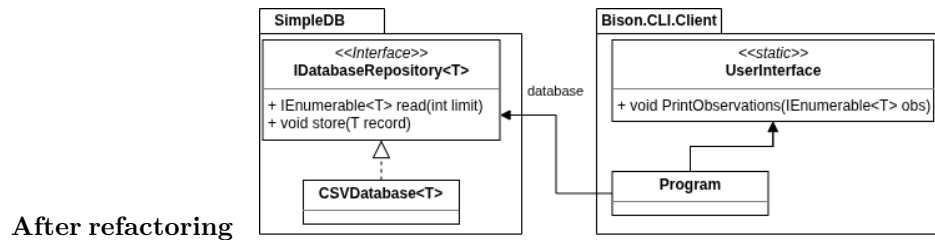


2.c) Refactor Bison.CLI's Program.cs so that all User Interface code resides in a separate class

Refactor all code that is concerned with printing cheeps to the terminal into a separate class called **UserInterface**. That is, after refactoring, your main program depends on this new class and calls a respective method whenever cheeps are printed to the terminal.

Likely, it is a good design to make the UserInterface and the contained methods **static**





2.d) Refactor Bison.CLI's Program.cs to use an external library CLI interface

Proper parsing of arguments and options that are provided to a program is difficult and error prone. Therefore, refactor your current `Program.cs` so that arguments passed to the program are not manually parsed out of the `args` property.

You may want to choose `System.CommandLine`, `CommandLine`, `docopt.net`, or any other comparable library that you find on nuget.org.

2.e) Add comments

So far, comments and observations are treated the same by Bison. Change this by separating these two concepts into two different classes and changing the interface. Do this task after you finished all other tasks. The observable behavior should be the following.

- An observation can be added with the `observe` command, and gets an id assigned once recorded. For example, use a simple counter.
- A comment can be added with the `comment` command and takes as a second parameter the id of the observation it comments. If no such observation exists, do not record the comment.
- The `read` command lists only observations
- Add a `discussion` command that takes an observation id as a parameter, and lists all comments for it

Internally, do not change the architecture, and use subclassing. Use two different CSV files.

After you added the new feature, did the refactoring of the previous task make this easier?

3) Ethics

Consider if want to share telemetry data with Microsoft. Per default, Microsoft collects data about many actions that you perform using their tools, see e.g., <https://learn.microsoft.com/en-us/dotnet/core/tools/telemetry> and <https://dotnet.microsoft.com/en-us/platform/telemetry>.

You can disable telemetry when using the `dotnet` command by creating an environment variable (in your `.bashrc` or whichever shell you are using) before invoking any `dotnet` commands:

- `DOTNET_CLI_TELEMETRY_OPTOUT=1` or

- `DOTNET_CLI_TELEMETRY_OPTOUT=true`

Additionally, VSCode collects telemetry data on how you are using it. You might want to disable that in the settings (set `telemetry.telemetryLevel` to `off`), see <https://code.visualstudio.com/docs/getstarted/telemetry>.

OBS: In general, when you build software in your later professional lives, it is a good practice to make collection of telemetry opt-in and not opt-out.